

Структура приложения

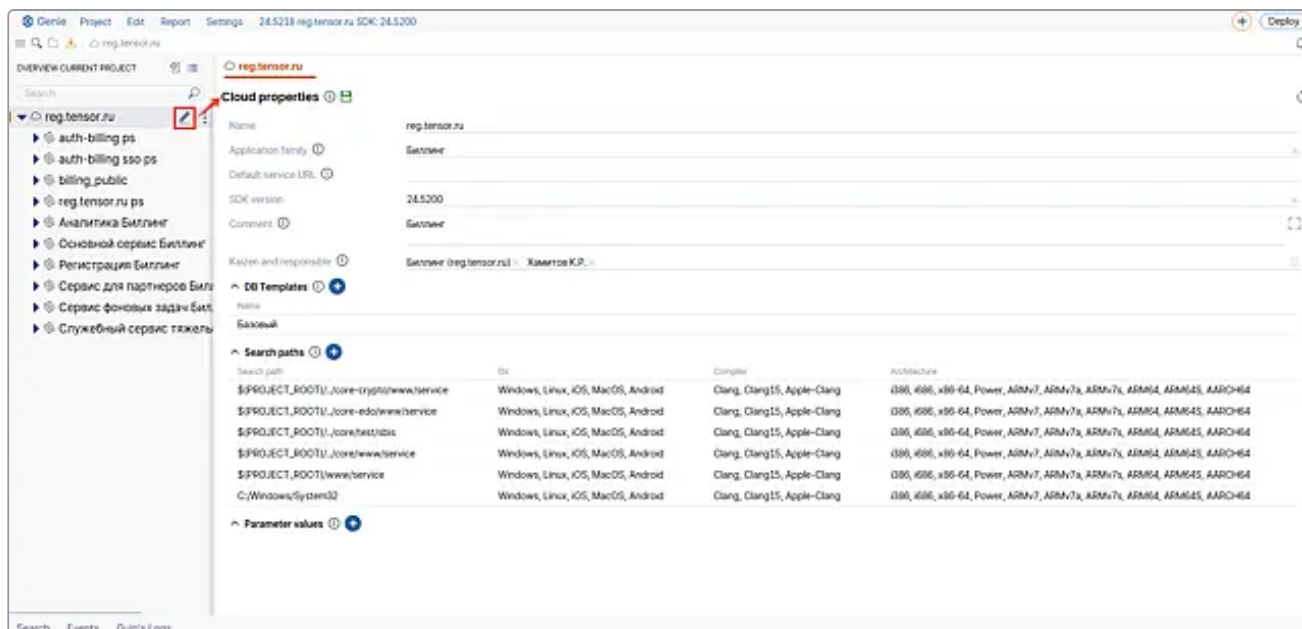
Разработка любых проектов на Wasaby Framework производится с помощью [IDE Genie](#). Для описания структуры проекта используется трёхуровневая иерархия типов файлов, где каждый следующий уровень описывает более низкоуровневый элемент проекта.

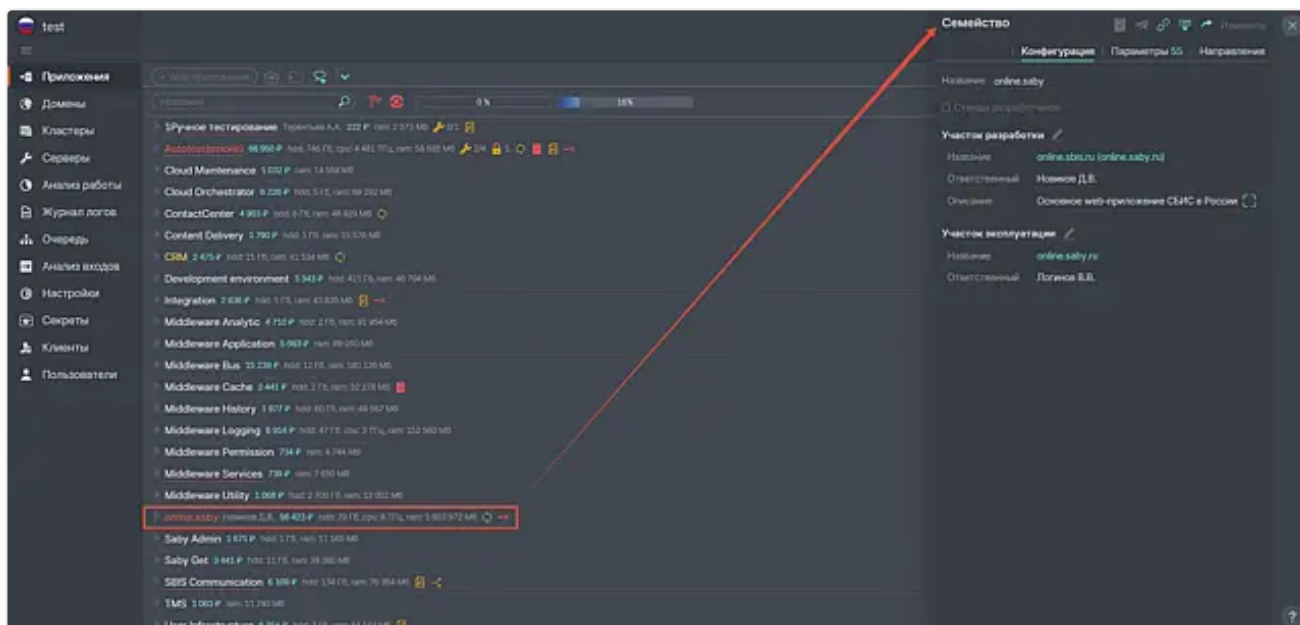
- **1-й уровень** – облако в целом, весь проект, сайт (см. [Облако, проект, приложение](#));
- **2-й уровень** – сервис приложения или сетевая служба, обеспечивающая обработку грс-запросов (см. [Сервис приложения](#));
- **3-й уровень** – сервисный или интерфейсный модуль (см. [Модуль](#)).

Файлы проекта идентифицируются посредством внутренней адресации, основанной на уникальных идентификаторах (UUID), что делает её устойчивой к переименованиям.

Облако, проект, приложение (s3cld)

Файл проекта приложения СБИС — это XML-файл с расширением **s3cld**, который формируется автоматически при создании нового проекта в Genie.





Добавление нового семейства приложений необходимо согласовать с [Новиковым Д.В.](#) через [Степанову Е.В.](#)

В s3cld-файле сохраняется в атрибуте "family_name" верхнего тега <cloud>.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <cloud family_name="online.saby">
3   ...
4 </cloud>
```

Версия **SBIS SDK**, совместимая с проектом (SDK version)

В s3cld-файле сохраняется в атрибуте "sdk_version" верхнего тега <cloud>.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <cloud sdk_version="25.4100">
3   ...
4 </cloud>
```

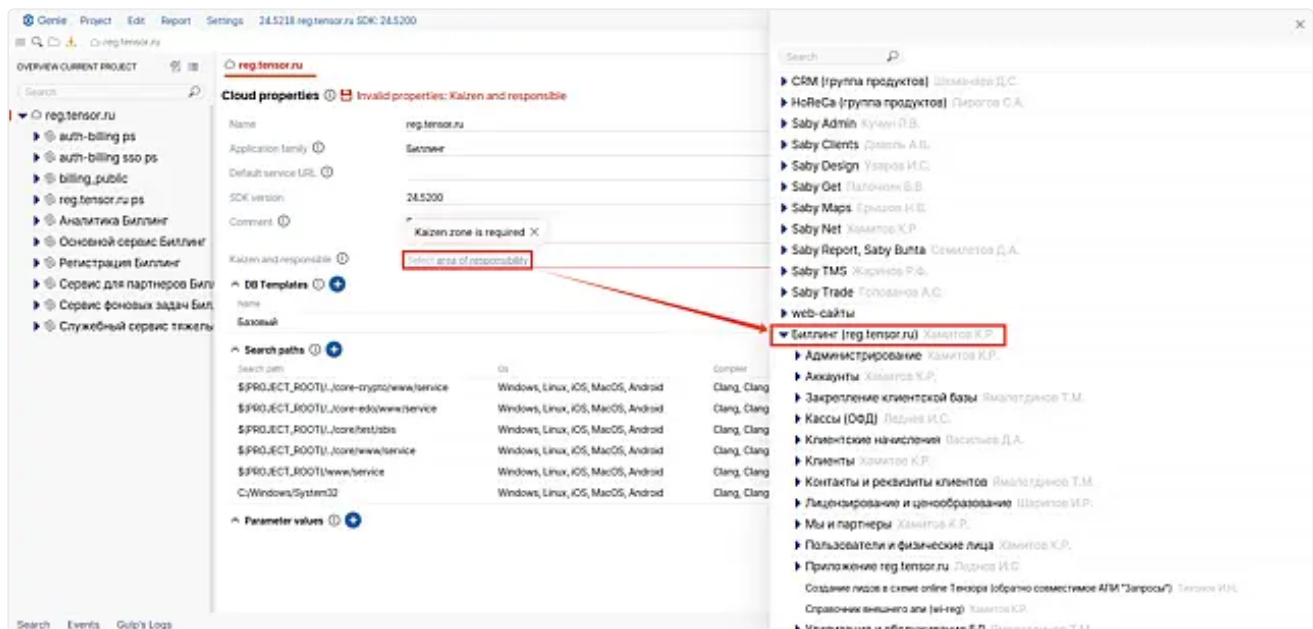
Участок Кайдзен и ответственный (Kaizen and responsible)

Выберите из справочника Кайдзен участок, который соответствует функциональности, реализованной в приложении. Как правило, это участок подсистемы СБИС, подробнее — в разделе "Подсистемы, сервисы и микросервисы" [Правил организации участков](#).

Важно!

В карточке приложения в реестре "Приложения" ответственный берется из описания участка приложения.

По умолчанию в качестве ответственного указывается ответственный за участок, но его можно сменить на другого участника команды участка.



В s3cld-файле [UUID участка](#) сохраняется в атрибуте "kaizen_zone" верхнего тега, а ответственный и его UUID - в атрибутах "responsible" и "responsible_uuid", соответственно.

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <cloud kaizen_zone="7204b7fc-b575-4851-9b0f-0a5443b3cae7"
3     responsible="Новиков Д.В."
4     responsible_uuid="9943efc7-7e01-473f-b80f-354fe7247359">
5     ...
6 </cloud>

```

Сервисы в составе приложения

Сервисы, созданные в проекте, сохраняются в секции <items> s3cld-файла.

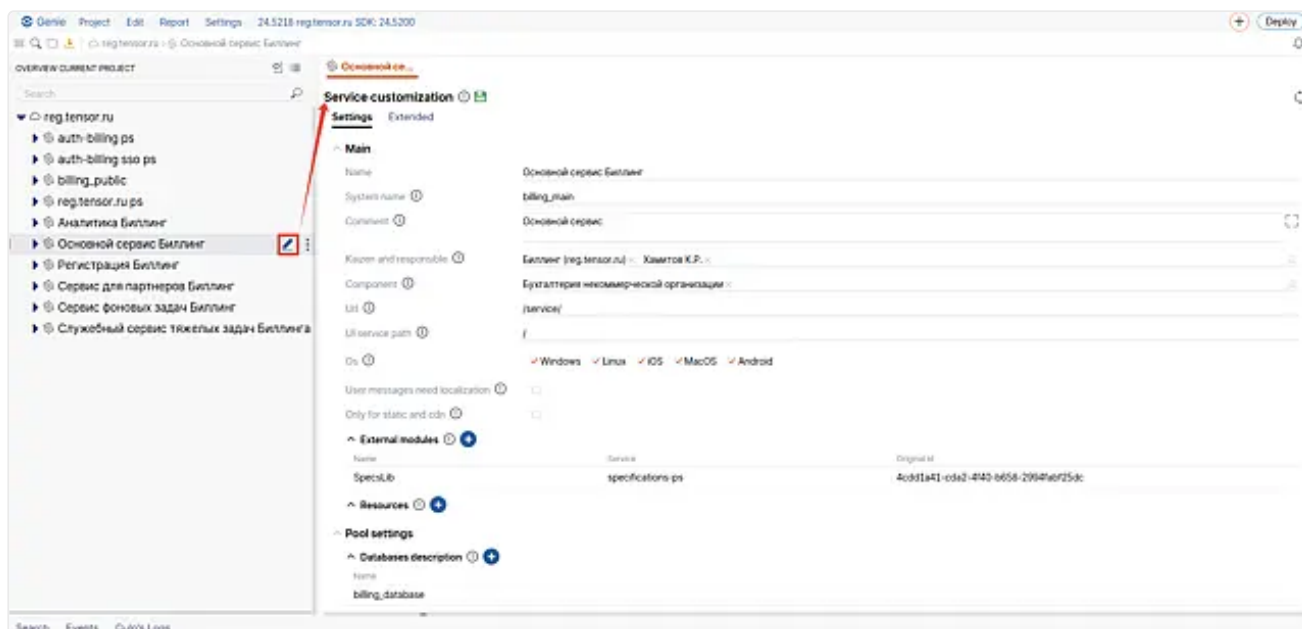
```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <cloud family_name="online.saby"
3     id="abe0b1aa-f2cc-44ba-8373-18865b607319"
4     name="online-ru"
5     platform="По умолчанию"
6     kaizen_zone="7204b7fc-b575-4851-9b0f-0a5443b3cae7"
7     responsible="Новиков Д.В."
8     responsible_uuid="9943efc7-7e01-473f-b80f-354fe7247359"
9     sdk_version="25.4100"
10    version="1.00">
11   <items>
12     <service id="d72c83dd-4c5e-4bb7-8262-19ed46c535d4"
13         name="online-ru" />
14     <service id="317d512c-17fe-4ace-8d87-c7735152fd8c"
15         name="online-ps-ru" />
16   </items>
17 </cloud>

```

Сервис приложения (s3srv)

Второй уровень проекта - сервис. Наведите на имя сервиса в дереве проекта и нажмите на значок "Properties" для настройки сервиса.



В состав проекта могут входить сервисы двух типов — **Сервис бизнес-логики** и **Сервис представления**.

Любой программный продукт содержит хотя бы один сервис.

Аналогично приложению, описание сервиса (его метаданные) сохраняются в XML-файле с расширением s3srv при конфигурировании сервиса в среде Genie.

Схема организации проектных файлов (дистрибутивов) основного сервиса приведена [здесь](#). Пояснения к схеме [описаны в Базе знаний](#).

Main

В данном разделе описываются основные настройки сервиса, а также указываются [внешние модули](#) и [дополнительные ресурсы](#).

Имя сервиса (Service name)

Имя, отображаемое в [Genie](#) и реестре "[Приложения \(Структура облака\)](#)". В s3srv-файле сохраняется в атрибуте "name" верхнего тега <service>.

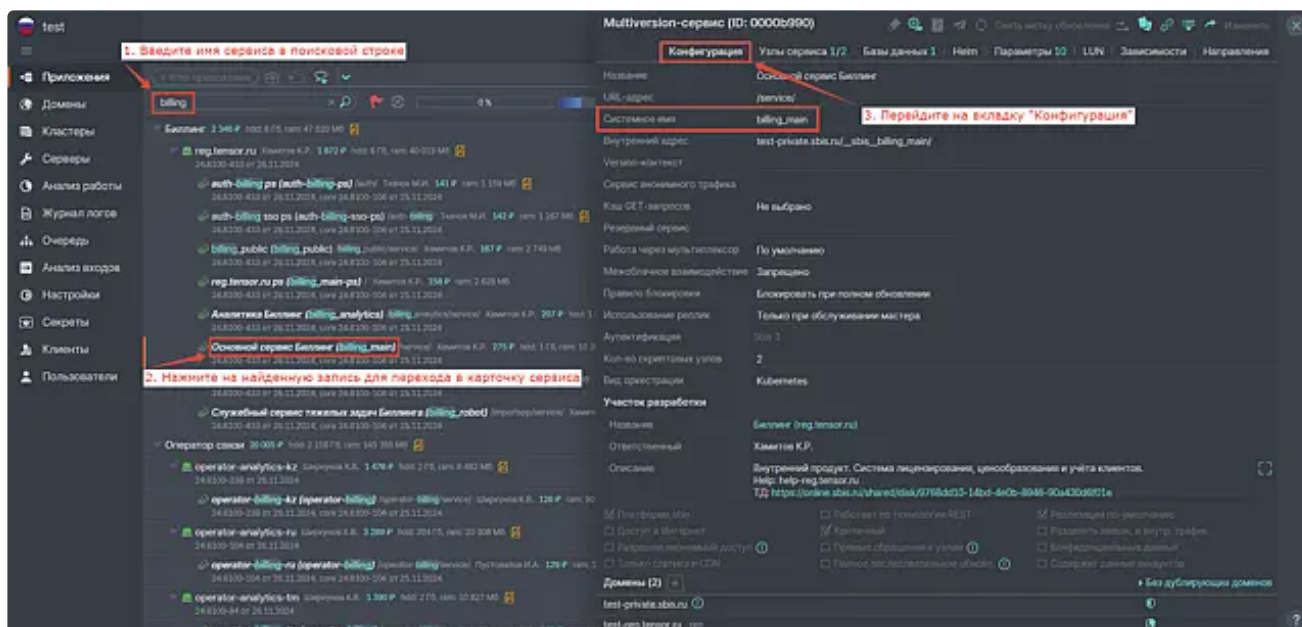
```
1 <service name="online-ru">
2   ...
3 </service>
```

Системное имя сервиса (System name)

Системное имя — это уникальное имя сервиса, по которому он идентифицируется в [облаке](#) среди других сервисов (отображается в реестре [Приложения \(Структура облака\)](#)). Имя должно соответствовать [стандарту именования сервисов](#).

Также системное имя используется при [развертывании приложения](#) на локальном стенде или сервере. Этим именем будет названа директория, в которой размещаются ресурсы сервиса.

Имя и системное имя отражаются в реестре "[Приложения \(Структура облака\)](#)". Можно найти интересующий сервис, чтобы посмотреть его карточку (конфигурацию, настройки и т.д.).



В s3srv сохраняется в атрибуте "system_name" верхнего тега <service>.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <service system_name="online">
3   ...
4 </service>
```

Comment

В поле Комментарий укажите краткое описание сервиса, для чего используется.

```
1 <service>
2   <description>Сервис исполнения распределенных сценариев ( графов
   выполнения задач БЛ )
3   Он предоставляет следующие возможности:
4   Ограничение нагрузки от определенного класса задач на узлы кластера
   согласно переданным параметрам.
5   Выполнение слияние задач в одну по установленным критериям.
6   Выполнение задачи отложено с задержкой.
7   Выполнение задач согласно установленным приоритетам.
8   Принятие и исполнение сценариев, которые состоят из вызова
   множества методов БЛ на разных узлах кластера.
9   Построение отчетов о текущем состоянии исполнения тех или иных
   задач.
10  </description>
11 </service>
```

Участок Кайдзен и ответственный (Kaizen and responsible)

Выберите из справочника Кайдзен участок, который соответствует функциональности, реализованной в приложении. Согласно разделу "Подсистемы, сервисы и микросервисы" [Правил организации участков](#) для сервиса должен быть создан участок, за исключением случаев, когда сервис реализует основную бизнес-логику или интерфейс подсистемы, в этом случае в качестве участка сервиса может быть указан участок подсистемы.



Важно!

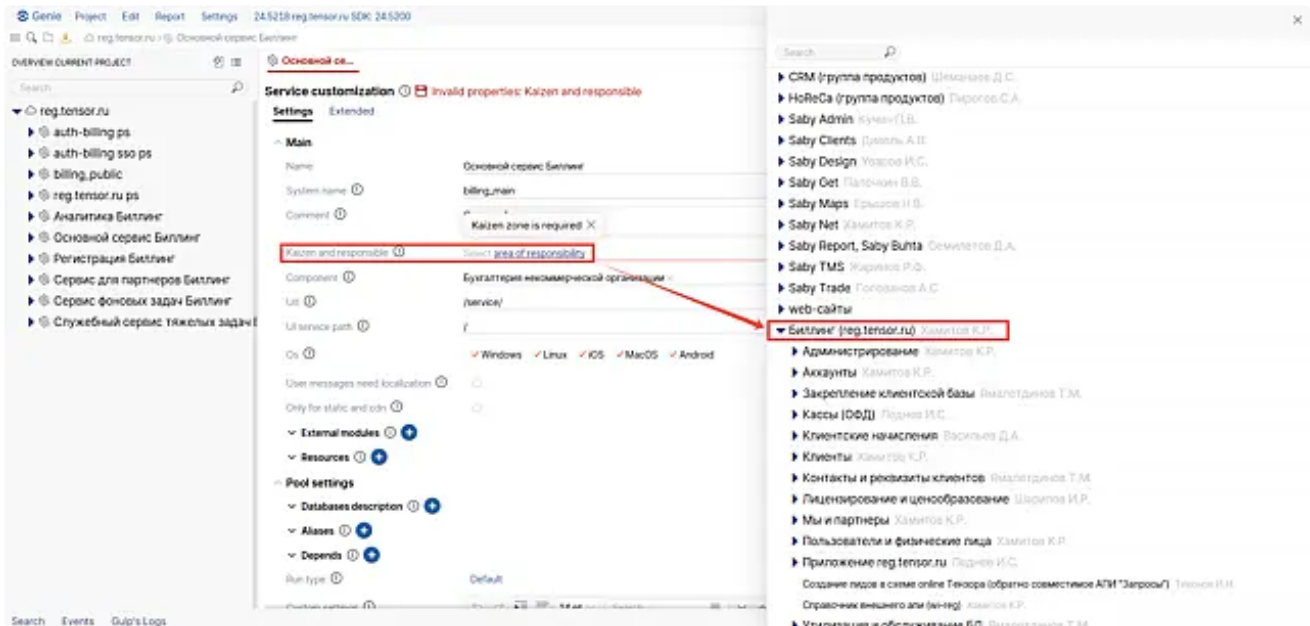
В карточке приложения в реестре "[Приложения \(Структура облака\)](#)" ответственный берется из описания участка приложения.

По умолчанию в качестве ответственного указывается ответственный за участок, но его можно сменить на другого участника команды участка.

```

1 <service kaizen_zone="448b4ae4-c2ae-11e3-bfd1-0050569139af"
2     ...
3     responsible="Хамитов К.Р."
4     responsible_uuid="65c05cc0-c9a9-41c2-a5cb-983da8e10e5b">
5 </service>

```



Component

Ссылка на компонент из [Маркета компонентов](#).

```

1 <service component_uuid="1061154b-9d7e-4a47-bb9f-38e21ec33c11">
2     ...
3 </service>

```

URL

Виртуальный каталог сервиса относительно корня домена.

```

1 <service url="/registration_service/service/">
2     ...
3 </service>

```

UI service path

Виртуальный каталог сервиса БЛ по умолчанию, на который отправляются запросы.

```

1 <service url="/registration_service/service/">
2     ...
3 </service>

```

Os

Операционные системы, с которыми совместим сервис.

User messages need localization

Признак того, что сообщения для пользователя, формируемые модулями сервиса, требуют перевода на язык пользователя. По умолчанию требование включено, но можно выключить. При включённом СТАН будет проверяться наличие переводов в таких файлах, как iux, flx, ogh.

Only for static and cdn

Модули из этого сервиса подлежат загрузке только на сервис статики или cdn. Признак того, что у этого сервиса не будет описания в структуре облака. Если в нем есть модули интерфейса, они будут грузиться только на сервис статики, а на сервис представления не будут.

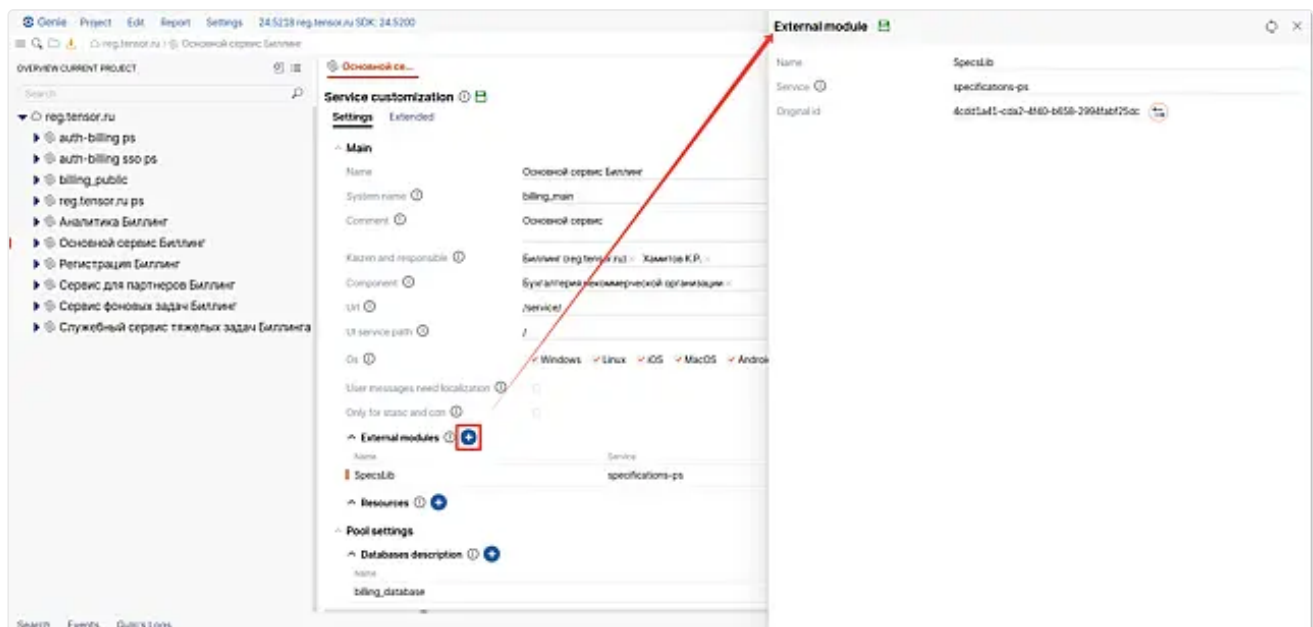
Внешние модули (External modules)

К сервису может быть подключен набор внешних (расположенных на других сервисах) модулей. Внешние модули включены в сервис непосредственно (т.е. не унаследованы от базовых сервисов). При добавлении указываются имя модуля (Name), системное имя сервиса (System name), на котором он расположен, а также уникальный идентификатор (Original id, указывается опционально).

При старте сервис получает id из сервиса "Управление облаком" по системному имени URL внешнего сервиса и его текущую версию. Полученные данные записываются в мета-файл описания модулей сервиса. По мере необходимости компоненты внешнего модуля подгружаются в сервис и исполняются в его контексте.

При обновлении сервиса, содержащего экспортируемый внешний модуль, производится уведомление всех связанных сервисов, которые, в свою очередь выполняют сброс кэша загруженного контента.

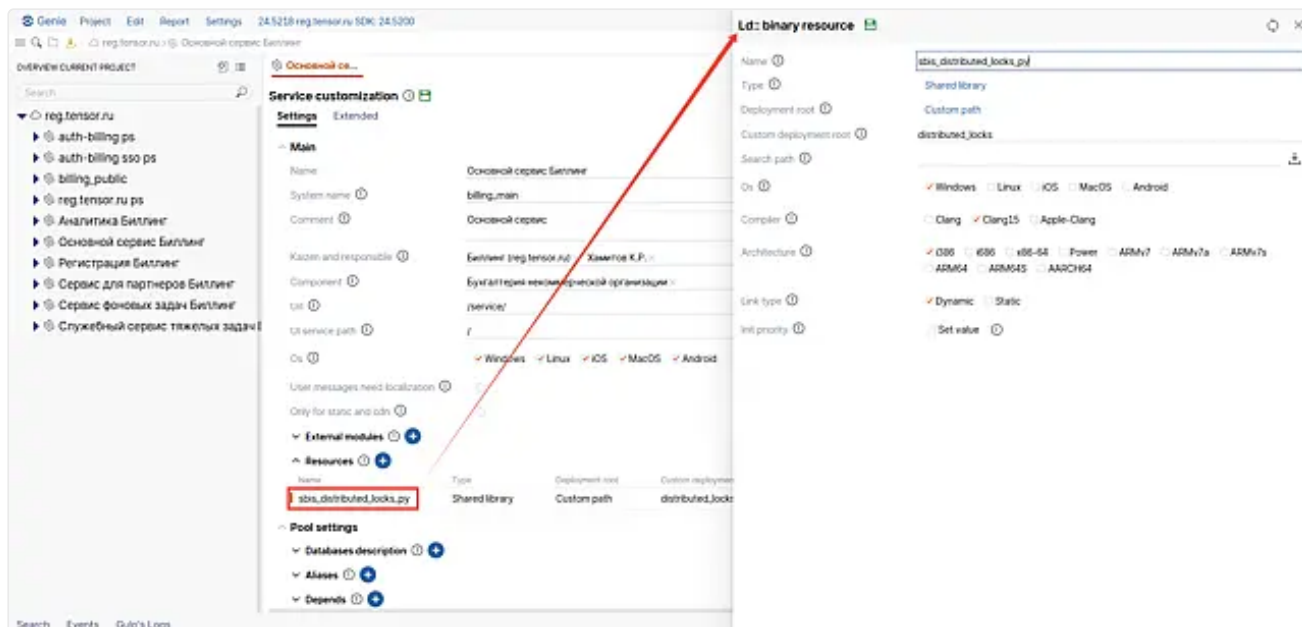
```
1 <service>
2   <items>
3     <external_module id="4cdd1a41-cda2-4f40-b658-2994fabf25dc"
4                           name="SpecsLib"
5                           original_id="4cdd1a41-cda2-4f40-b658-
6 2994fabf25dc"
7                           service="specifications-ps"/>
8   </items>
9 </service>
```



Ресурсы модулей и сервисов (Resources)

Ресурсы сервисов - это файлы, директории, разделяемые библиотеки или исполняемые файлы, которые необходимы сервису. В Genie ресурсы отображаются в разделе **Resources**.

```
1 <service>
2   <resources>
3     <resource architecture="all"
4                   compiler="all"
5                   custom_deployment_root="ui"
6                   deployment_root="custom_path"
7                   name="ver.html"
8                   os="all"
9                   type="file"
10                  version="all"/>
11   </resources>
12 </service>
```



Name

Название ресурса.

```
1 <service>
2   <resources>
3     <resource name="ver.html" />
4     ...
5   </resources>
6 </service>
```

Type

Тип ресурса. Вместе с названием однозначно определяют файл, который будет добавлен к ресурсам сервисного модуля.

- Shared library - добавление [разделяемой библиотеки](#). Например, для платформы Windows это могут быть DLL-файлы (.dll), для Linux - .so, для macOS - *.dylib. Подробнее о разделяемых библиотеках читайте в статье [Библиотека \(программирование\)](#).
- Executable file - добавление исполняемого файла. Подробнее о них, а также полный список расширений исполняемых файлов, смотрите в статье [Исполняемый файл](#).
- Folder - добавление директории целиком.
- File - добавление файла, расширение которого не относится к другим существующим значениям этой опции. Например, в качестве файлов можно добавлять изображения.
- Python library - добавить стороннюю библиотеку. Подробнее о подключении сторонних библиотек Python описано [здесь](#).

Deployment root

Директория, в которой будет размещен подключаемый ресурс после развертывания приложения (None, Module root, Service root, Custom path).

```
1 <service>
2   <resources>
3     <resource deployment_root="custom_path" />
4     ...
5   </resources>
6 </service>
```

При выборе значения Custom path отображается опция **Custom deployment root**, которая применяется, когда при развертывании приложения требуется разместить подключаемые дополнительные ресурсы не в корне текущего сервисного модуля.


```

1 <service>
2   <resources>
3     <resource custom_deployment_root="./." />
4     ...
5   </resources>
6 </service>

```

Search path

Путь к директории на жестком диске, в которой нужно производить поиск ресурса.

Допускается использование двух форматов пути:

- относительно каталога с файлом проекта (s3cld): \$(PROJECT_ROOT)/../dir1/
- относительно каталога с текущим файлом: ../../dir1/

Нет смысла размещать дополнительные ресурсы в корне БЛ-модуля, так как все, что находится в каталоге с модулем, принадлежит ему безусловно и попадет в результирующую структуру каталогов вне зависимости от конфигурации ОС, компилятора, сборки и архитектуры.

```

1 <service>
2   <parent id="a5bf0cbf-f412-436e-9155-df1f81d55ec3"
3     name="basic_platform"
4     path="$(SBISPlatformSDK)/components/basic_platform.s3srv" />
5 </service>

```

Os

Некоторые элементы дерева проекта могут быть платформозависимыми. Платформозависимый [сервис приложения](#) (s3srv-файл) существует не под все операционные системы, соответственно, в описании сервиса в Genie, в атрибуте Os активированы не все флаги, в зависимости от операционной системы.

Платформозависимый сервис нужно разворачивать в составе приложения или использовать в качестве родительского сервиса для других сервисов только в случае, если развертывание ведется на одной из поддерживаемых операционных систем.

[Модуль](#) также может быть платформозависимым. При развороте сервиса на операционную систему, не указанную в описании модуля, такой модуль будет отсутствовать.

Платформозависимые ресурсы имеют более богатую конфигурацию, т.е. такие ресурсы могут быть нужны не только под определенную операционную систему, но и под определенные компилятор сборки и архитектуру. Это значит, что такой ресурс должен быть упакован в состав приложения, когда сборка осуществляется под набор конфигураций, соответствующих этой маске.

```

1 <service>
2   <resources>
3     <resource os="windows/linux" />
4   </resources>
5 </service>

```

Compiler

Компилятор:

- Clang - высокопроизводительный и точный компилятор для языков программирования C, C++, Objective-C и CUDA, а также набор инструментов для анализа кода
- Clang15 - версия Clang, поддерживаемая ранее.
- Apple-Clang - компилятор C, C++ и Objective-C, основанный на проекте LLVM, который Apple использует как основной компилятор в операционных системах macOS, iOS и других.

```

1 <service>
2   <resources>
3     <resource compiler="clang15" />
4     ...
5   </resources>
6 </service>

```

Architecture

Архитектура процессора.

- i386 - 32-разрядные процессоры архитектуры Intel 1988 года.
- i686 - 32-разрядные процессоры архитектуры Intel.
- x86_64 - 64-разрядные процессоры архитектуры Intel.
- PPC64 - процессоры архитектуры PowerPC.
- ARM* - процессоры архитектуры ARM.

```
1 <service>
2   <resources>
3     <resource architecture="i386" />
4     ...
5   </resources>
6 </service>
```

Link type

Тип линковки выбирается в зависимости от требуемого ресурса.

- Dynamic - динамические библиотеки.
- Static - статические библиотеки.

```
1 <service>
2   <resources>
3     <resource link_type="dynamic" />
4     ...
5   </resources>
6 </service>
```

Init priority

Приоритет загрузки (необязателен). Если указан, то ресурс будет загружен в процесс как разделяемая библиотека средствами Платформы.

```
1 <service>
2   <resources>
3     <resource init_priority="1" />
4     ...
5   </resources>
6 </service>
```

Pool settings

Настройки пула сервисов. Подробнее читайте в [статье](#).

Наследование сервисов

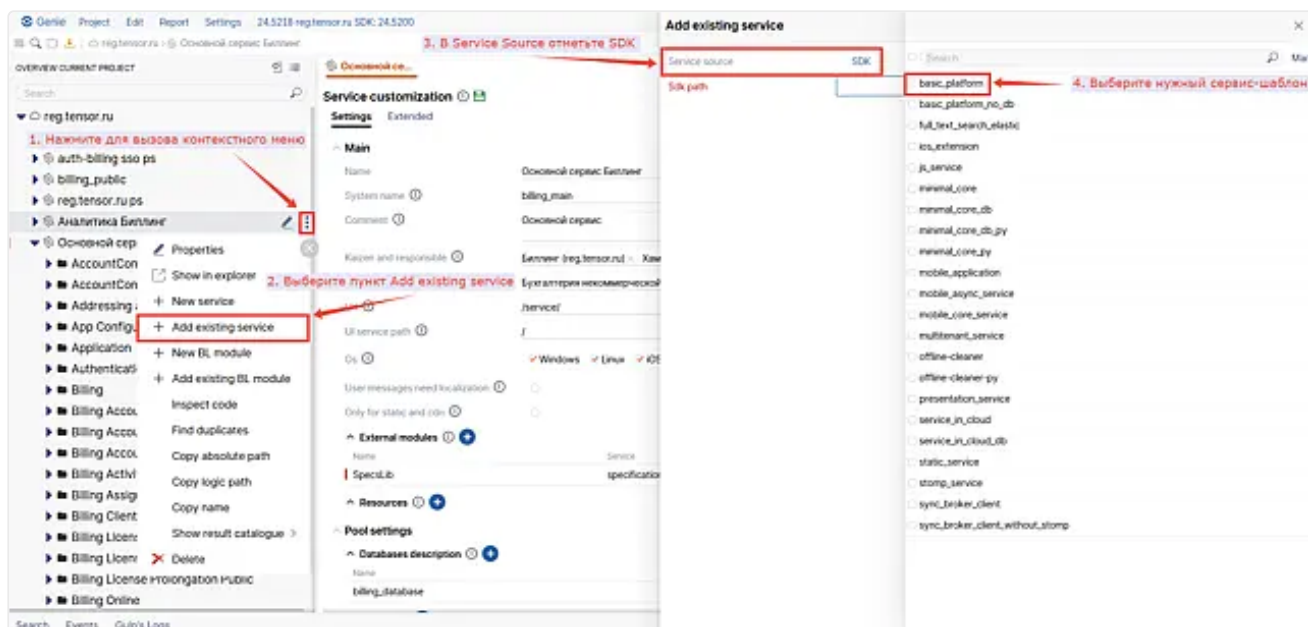
Для отслеживания отношений вида "Сервис X является расширением (специальной конфигурацией) сервиса Y" между сервисами бизнес-логики было введено понятие наследования сервисов друг от друга.

На прикладном уровне фраза "Сервис X наследуется от сервиса Y" означает, что сервис X, кроме своих собственных модулей БЛ, будет включать также полный набор модулей БЛ от базового сервиса Y (то есть будет расширять его).

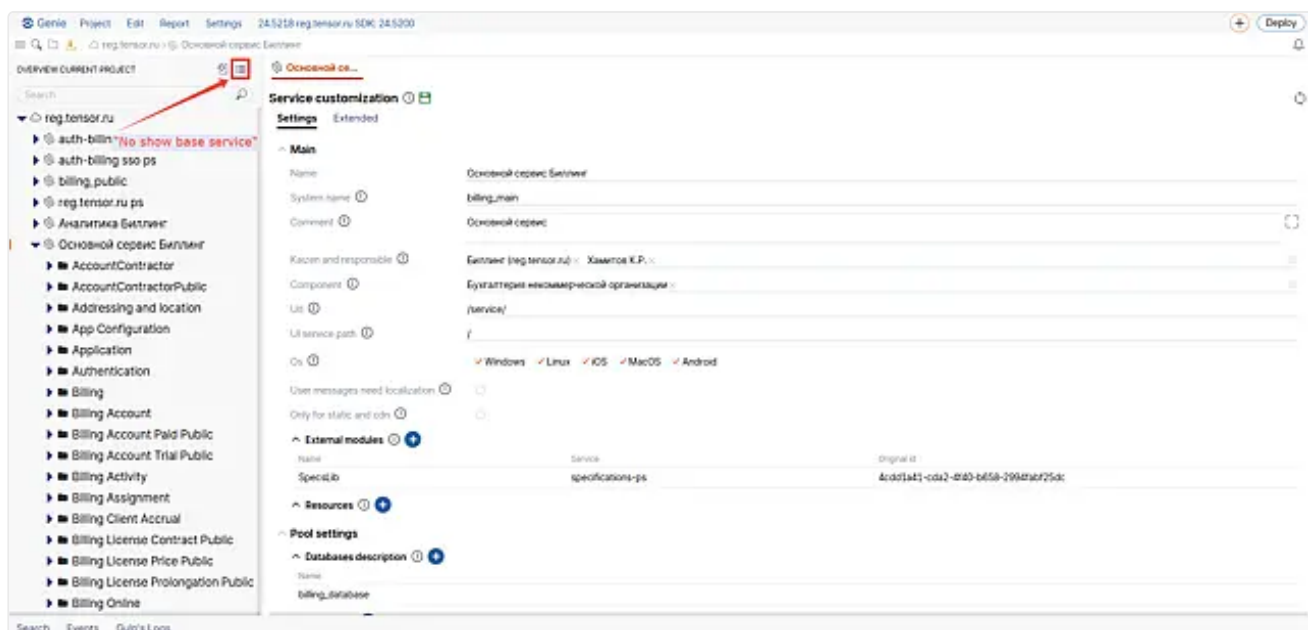


Наследование сервисов играет важную роль при [описании API](#) и внешних вызовов модулей БЛ, о чем будет сказано далее.

Добавление наследуемого сервиса выполняется с помощью контекстного меню сервиса-родителя.



Также наследуемые сервисы можно скрыть из дерева с помощью значка "No show base services".



Сервис, от которого следует производить наследование, не обязательно должен входить в проект Genie.

Сервисы выстраивают некоторую цепочку иерархии, при этом самый дочерний сервис будет включать в себя все модули предыдущих сервисов вплоть до самого базового.

Если сервис X уже содержал в себе модуль A, и наследуется от сервиса Y, который тоже содержит модуль A, то при наследовании сервис X будет считать модуль A как свой собственный, а не унаследованный, но если его удалить, модуль A не исчезнет из сервиса, а помечится как "унаследованный".



При этом модули, которые пришли из базового сервиса, невозможно переместить либо удалить. При попытке это сделать будет выдано соответствующее предупреждение. Если, например, удалить или переместить модуль из базового сервиса, то в дочернем он тоже автоматически удалится, и изменения синхронизируются.

Модуль (s3mod)

Модуль — это логически и функционально завершённый код, который имеет минимально возможное количество зависимостей от других модулей.

Модули, в зависимости от назначения, делятся на **сервисные**, которые включены в сервис бизнес-логики (БЛ), и **интерфейсные (GUI)**, составляющие сервис представления (СП).

Следует отметить, что такое разделение модулей — несколько условное. В рамках разработки продуктов СБИС выработалось правило использовать JavaScript для разработки GUI-модулей, а C++ и Python — для сервисных

модулей.

Однако по ряду причин модули, содержащие js-код, также используются для реализации функционала на сервере.

Документ не ставит целью перечисление всех возможных причин подобной реализации. Для понимания проиллюстрируем две:

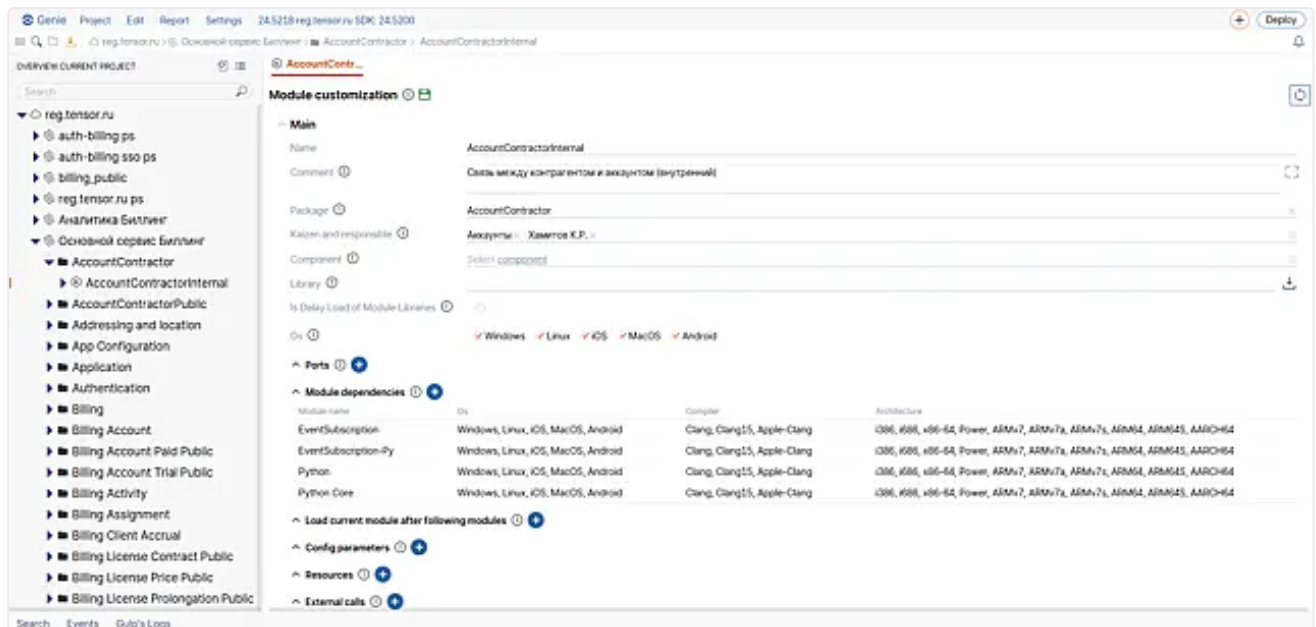
- решение проблемы дублирования кода или исключение двойной реализации — в БЛ на Python и в СП на JavaScript;
- передача большого объема информации между БЛ и СП. В качестве примера можно привести реализацию функционала ФЭД (Формализованных Электронных Документов). Здесь js-модули, включенные в БЛ, используются с целью уменьшения трафика между СП и БЛ.

Общие правила организации модулей

1. Не должно быть "царь-модулей", в которых собран целый продукт или весь код сотрудника.
2. Если в модуле ресурсы относятся к разным компонентам, то это признак, что модуль слишком крупный и его нужно разбить.
3. У модуля должны быть указаны ВСЕ зависимости. Если с этим трудности, то возможно у "соседних" модулей проблема, обозначенная в п.2.

Основные атрибуты модулей

Аналогично приложению и сервису, описание модуля (его метаданные) сохраняются в XML-файле с расширением **s3mod** при конфигурировании сервиса в среде Genie.



Comment

Пользовательский комментарий.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3   <description>В модуле реализована поддержка работы с функционалом.
4   </description>
5 </bl_module>
```

Package

Пакет, предназначен для визуальной группировки модулей внутри одного сервиса или интерфейса приложения.

Правила объединения модулей в пакеты описаны [здесь](#).

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module
3   package="Application">
4   ...
```

5 </bl_module>

Kaizen and responsible

Участок Кайдзен и ответственный за модуль. В Genie указывается участок Кайдзен, который соответствует функциональности, реализуемой в модуле.

По умолчанию в качестве ответственного за модуль указывается ответственный за участок. Его можно сменить на другого участника команды участка. Ответственный за модуль не из команды участка считается невалидным.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module
3     kaizen_zone="d1f38d58-86d3-42e3-a05a-71dbcf80783f"
4     responsible="Терентьев А.А.">
5     ...
6 </bl_module>
```

Component

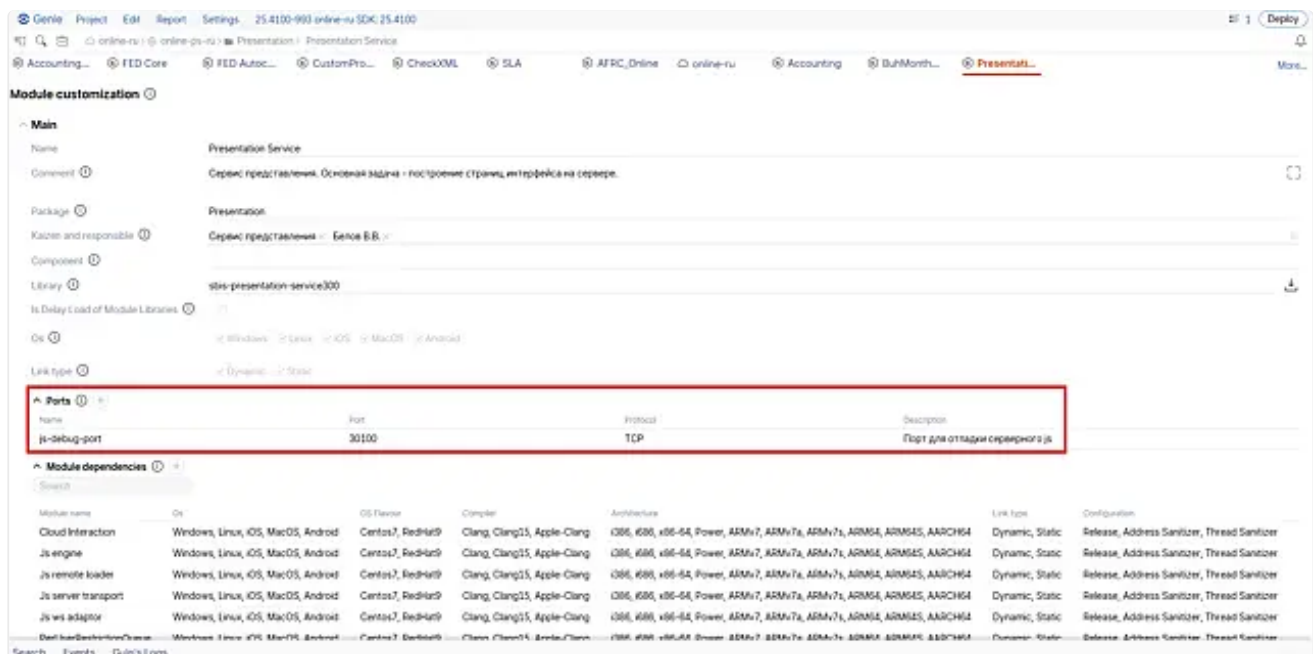
В этом атрибуте указывается идентификатор компонента системы, функциональность которого реализована в модуле, причем это должен быть низший по иерархии подходящий компонент.

Реестр компонентов можно посмотреть в разделе Компоненты приложения Saby Inside.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module
3     component_uuid="34b04616-689e-4e6f-8a7b-a0649f61a475">
4     ...
5 </bl_module>
```

Порты

Список дополнительных портов, прослушиваемых сервисом.



Name	Port	Protocol	Description
js-debug-port	30100	TCP	Порт для отладки серверного js

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3     ...
4     <ports>
5         <item description="Порт для отладки серверного js"
6             name="js-debug-port"
7             port="30100"
8             protocol="TCP"
9             type="port" />
```

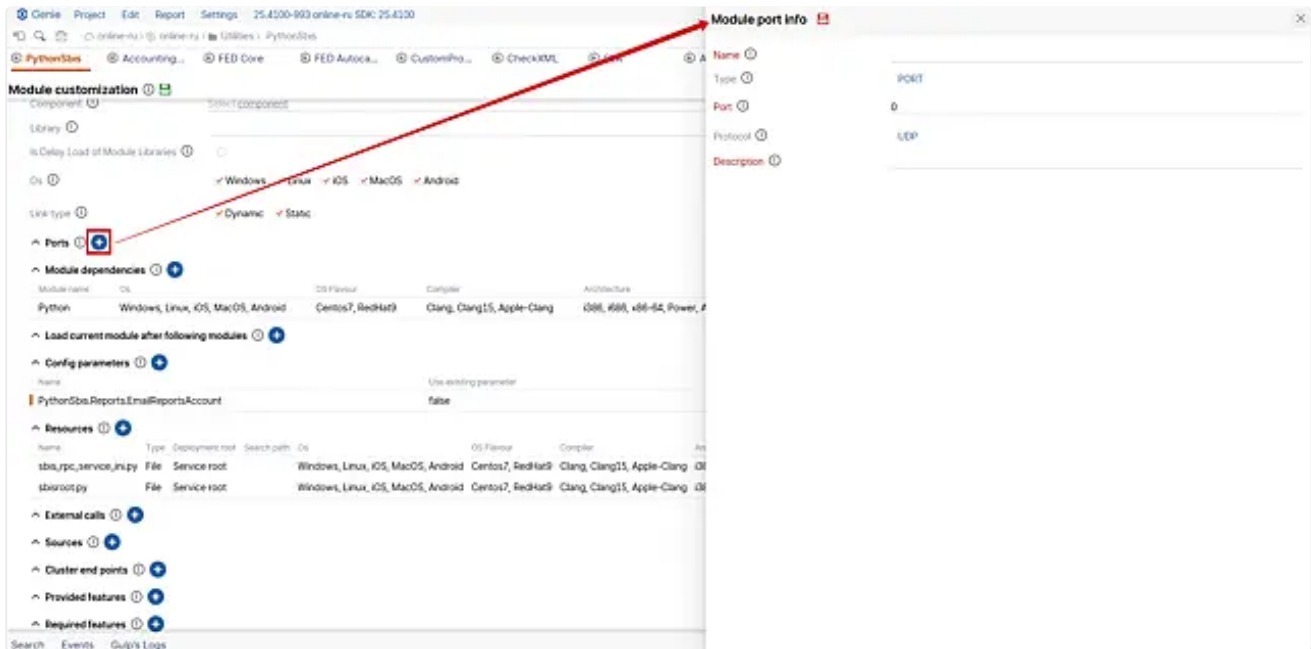
```

10 </ports>
11 </bl_module>

```

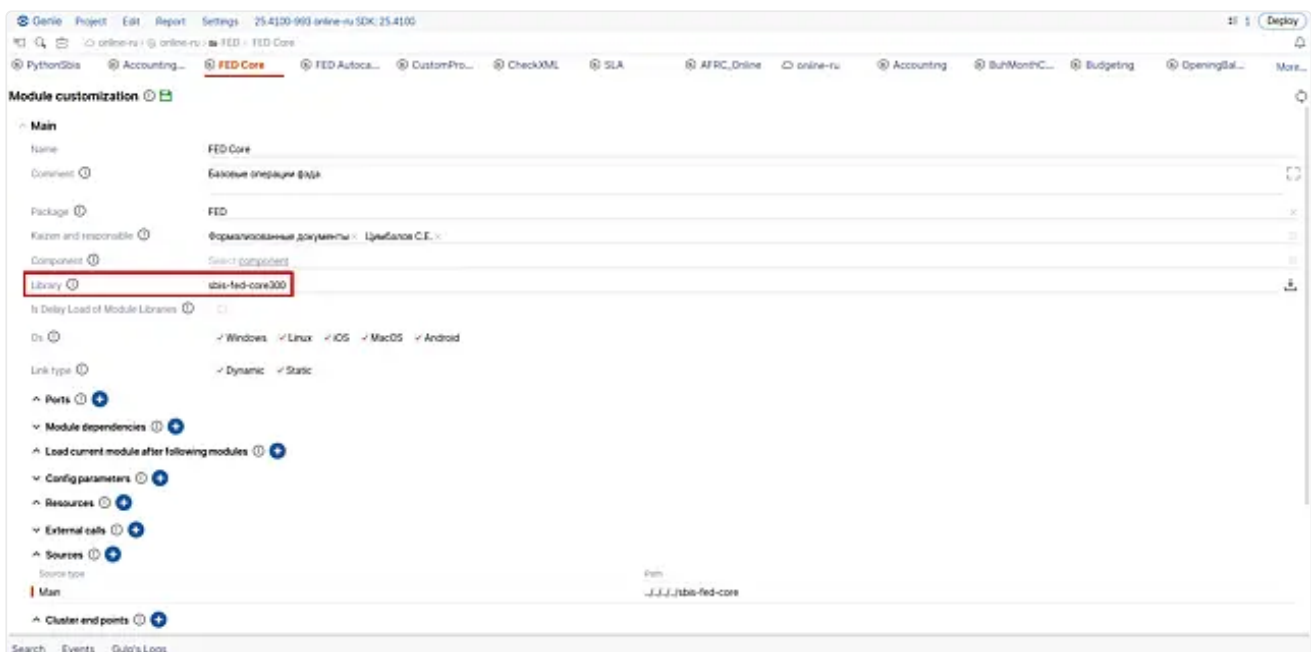
Для каждого порта можно задать следующие атрибуты:

- **Name** - имя порта, указывается латиницей. В k8s данному порту будет присвоена метка с этим именем.
- **Type** - формат значения: порт (PORT) или диапазон портов (RANGE).
- **Port** - номер порта, целое число в интервале от 1024 до 65535.
- **Protocol** - используемый протокол (UDP, TCP, HTTP или HTTPS).
- **Description** - комментарий.



Библиотеки

В библиотеках хранятся файлы модуля. Они не описываются в Genie и хранятся в папке модуля. Например, для сервисного модуля **FED Core**:



```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module
3     bl_module_library="sbis-fed-core300">
4     ...
5 </bl_module>

```


Если модуль предоставляет в своем составе компилируемые артефакты (одну или несколько библиотек, либо исполняемые файлы), то библиотека должна ссылаться на [исходники этих артефактов](#).

Зависимости модулей

Приложения на платформе Sabu построены по плагиновой архитектуре. Функционал предоставляется небольшими частями, которые называются модулями. Модули могут находиться друг от друга в зависимости т. е. модуль может нуждаться для корректной работы в ресурсах другого модуля. Некоторые зависимости могут быть вариативны из-за особенностей функционала приложения или специфики платформы. Правильный отбор модулей и установка зависимостей является необходимым условием для корректной сборки и функционирования приложения.

Существуют следующие типы зависимостей между модулями:

1. **Module dependencies** — прямые зависимости. Этот тип зависимости означает, что данный модуль не работоспособен без указанных в качестве зависимостей модулей. Например, он использует классы, реализованные в модуле зависимости. Если прямые зависимости одного из модулей не удовлетворены, то происходит ошибка загрузки.

Можно подключать только зависимости из s3srcv-файлов, причем каскадные зависимости не нужно подключать явно, достаточно корневой.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3   <depends>
4     <module id="c958547e-5272-45fd-b723-af30678b7340" name="BL
Core"/>
5   </depends>
6 </bl_module>
```

Пусть у модуля А есть прямая зависимость от модуля В. Тогда сначала будет загружен модуль В, а затем модуль А. Если в составе сервиса не окажется модуля В, но есть модуль А, то это приведет к отказу старта сервиса.

2. **Load current module after following modules** — косвенные зависимости. Несмотря на то, что указанный модуль не находится в прямой зависимости, он должен загружаться после него. Если модули окажутся в составе одного сервиса, то данный модуль будет загружен после указанного. Как правило, это применяют для реализации перекрытия части методов бизнес логики. Требуется крайне редко. Если модуля зависимости нет в составе сервиса, то это не приведёт к ошибке загрузки. Прямые зависимости и косвенные зависимости могут быть платформозависимыми.

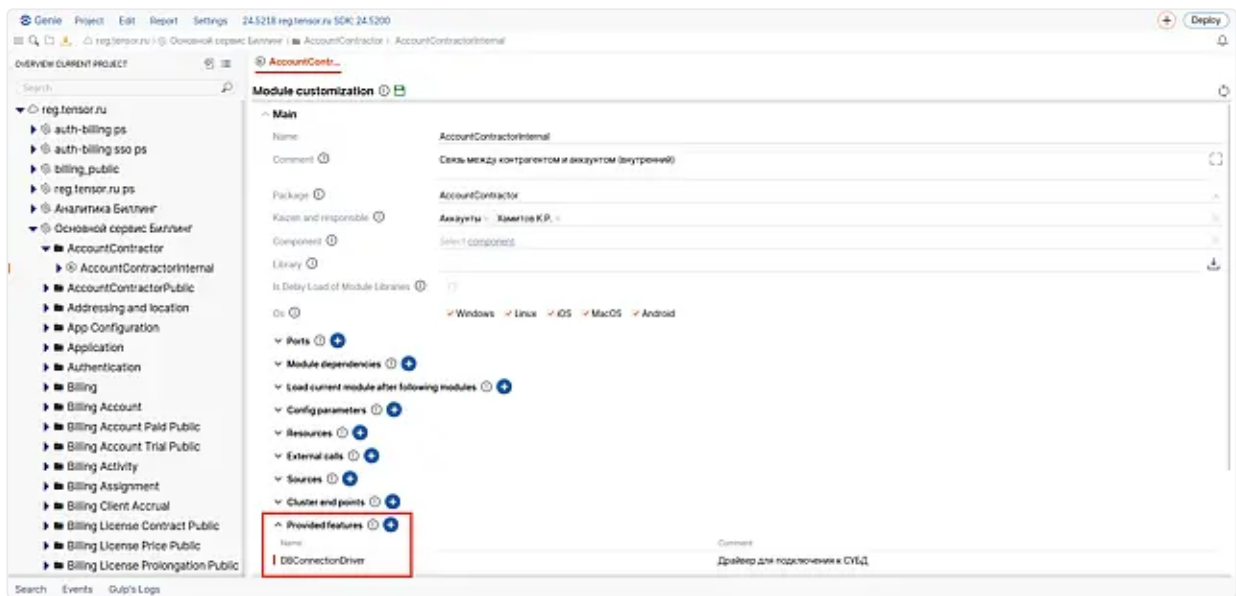
```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3   ...
4   <load_after>
5     <bl_module
6       id="..."
7       name="MobileModuleAmalgamation"/>
8   </load_after>
9   ...
10 </bl_module>
```

Пусть у модуля А есть косвенная зависимость от модуля В. Тогда, при наличии в составе сервиса обоих модулей, сначала будет загружен модуль В, а затем модуль А. Однако, если в составе сервиса модуля В не окажется вовсе, сервис все равно запустится и будет работать.

3. **Зависимости открытого типа**. Данный вид зависимостей реализует модель "Поставщик - потребитель". "Потребитель" требует, чтобы в составе конечного приложения была поставлена реализация некоторого функционала, но не знает конкретного "поставщика". Например, модуль определяет некий интерфейс и ожидает, что он будет кем-либо реализован.

Данный механизм реализован через атрибуты:

- **Provided features** - задаётся для "поставщика" (модуля, компонента).

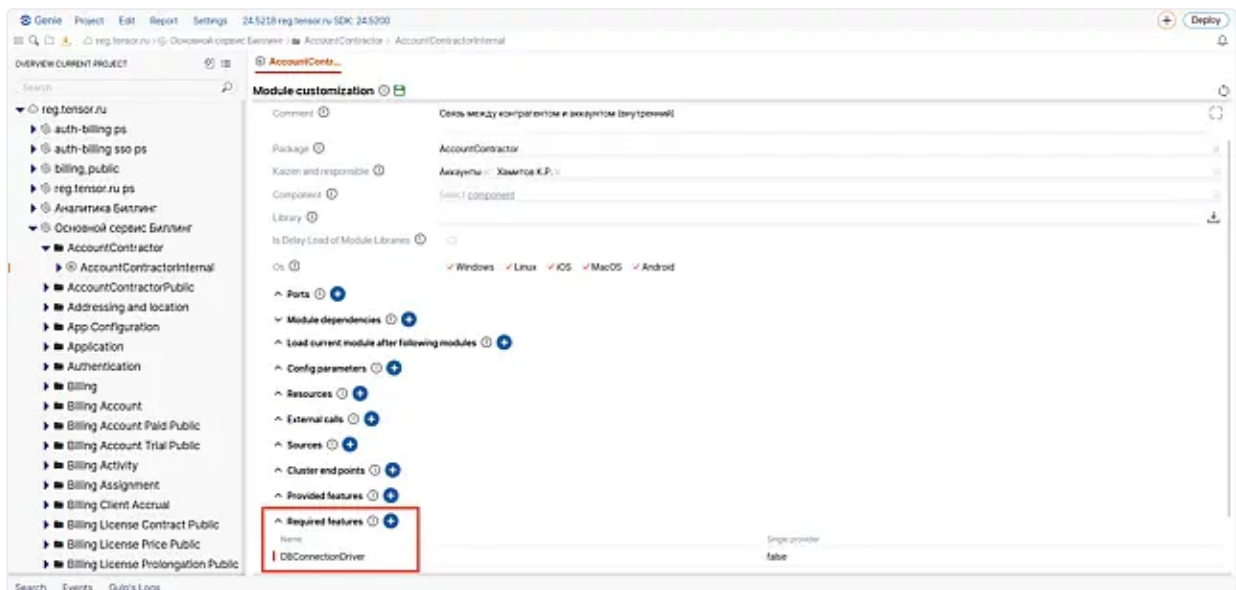


```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3   ...
4   <features_required>
5     <feature name="DBConnectionDriver"/>
6   </features_required>
7   ...
8 </bl_module>

```

- **Required features** - задаётся для компонентов, предоставляемых текущим модулем.



```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3   ...
4   <features_provided>
5     <feature name="DBConnectionDriver"/>
6   </features_provided>
7   ...
8 </bl_module>

```

Например, модуль **BL Async** из состава SDK платформы предоставляет функционал, необходимый для использования асинхронных вызовов. Для реализации данного функционала ему необходимо размещать принятые асинхронные вызовы в некотором локальном персистентном хранилище. Данное хранилище объявлено как интерфейс, но что это за хранилище для "потребителя" не важно. В списке зависимостей модуля **BL Async** указан компонент **ImplStorageForAsyncCalls**. "Потребитель" может потребовать, чтобы функционал был реализован только одним "поставщиком", для этого установите флаг **Single provider**.

Также в составе SDK поставляются модули **Berkeley Storage** и **SQLite Storage**. Berkeley Storage используется в серверных решениях, а SQLite Storage в offline-приложениях. Данные модули поставляют требуемый функционал указанному модулю.

Параметры конфигурации

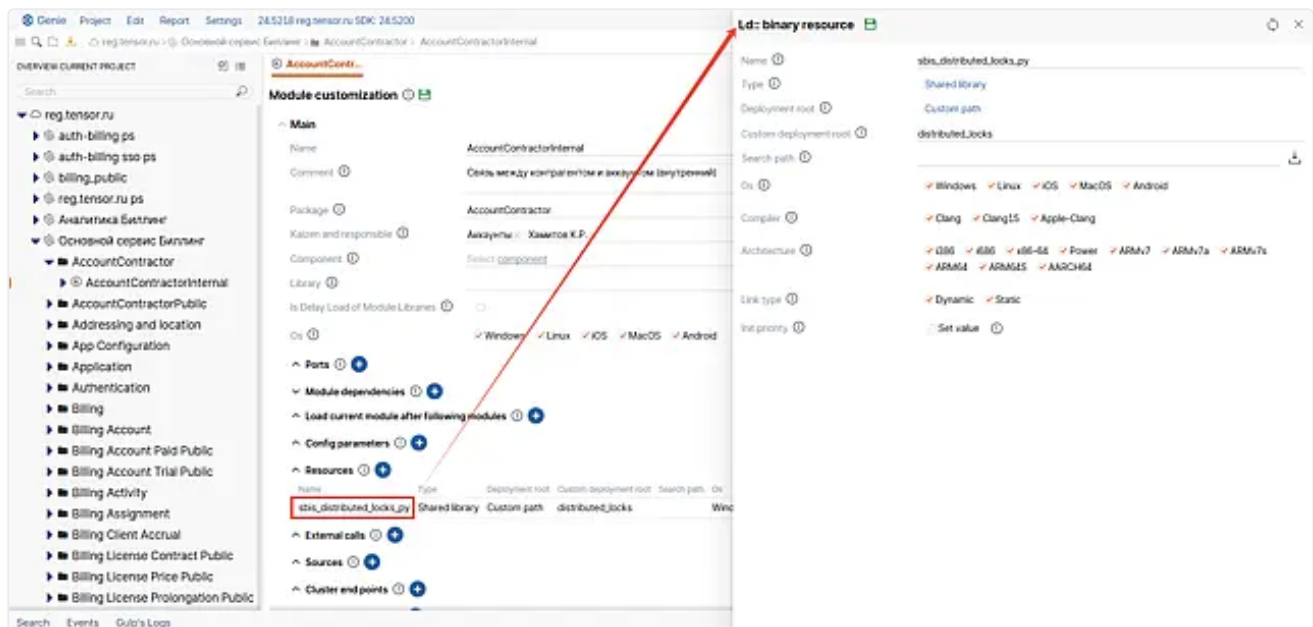
Используются при разворачивании отладочного стенда: со всего облака для каждого сервиса собираются параметры, которые далее записываются в конфигурационный файл сервиса (sbis-rpc-service.ini) вместе со значениями по умолчанию или значениями, указанными вручную. Также их значения можно использовать в прикладном коде с помощью синглтона sbis::Config.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3     ...
4     <parameters>
5         <parameter
6             db_deployment_enabled="1"
7             full_path_address="1"
8             name="Ядро.Асинхронные сообщения.AMQPTimeout"
9             required_debug="0"
10            required_release="0"
11            subgroup_name="Прочие"
12            type="number"
13            unit="мс"
14            version="3">
15            <comment>
16                <![CDATA[Определяет таймаут при обращении к rabbitmq.
17                При превышении таймаута будет брошено исключение.
18                Минимально допустимый таймаут - 50 мс.]]>
19            </comment>
20            <default configuration="debug">
21                <![CDATA[10000]]>
22            </default>
23            ...
24        </parameter>
25        <parameter ...>
26            ...
27        </parameter>
28    </parameters>
29    ...
30 </bl_module>
```

Подробнее о параметрах конфигурации сервисного модуля описано [здесь](#).

Ресурсы модулей

Ресурсы модулей - это файлы, директории, разделяемые библиотеки или исполняемые файлы, которые необходимы модулю. В Genie ресурсы отображаются в разделе [Resources](#) и имеют же атрибуты, что и у сервисов.



```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <bl_module ...>
3      ...
4      <resources>
5          <resource
6              architecture="all"
7              compiler="all"
8              custom_deployment_root="core_sbis_blservice"
9              deployment_root="custom_path"
10             name="sbisblservice"
11             os="windows/linux/macos"
12             type="shared_library"
13             version="all"/>
14      </resources>
15      ...
16  </bl_module>

```

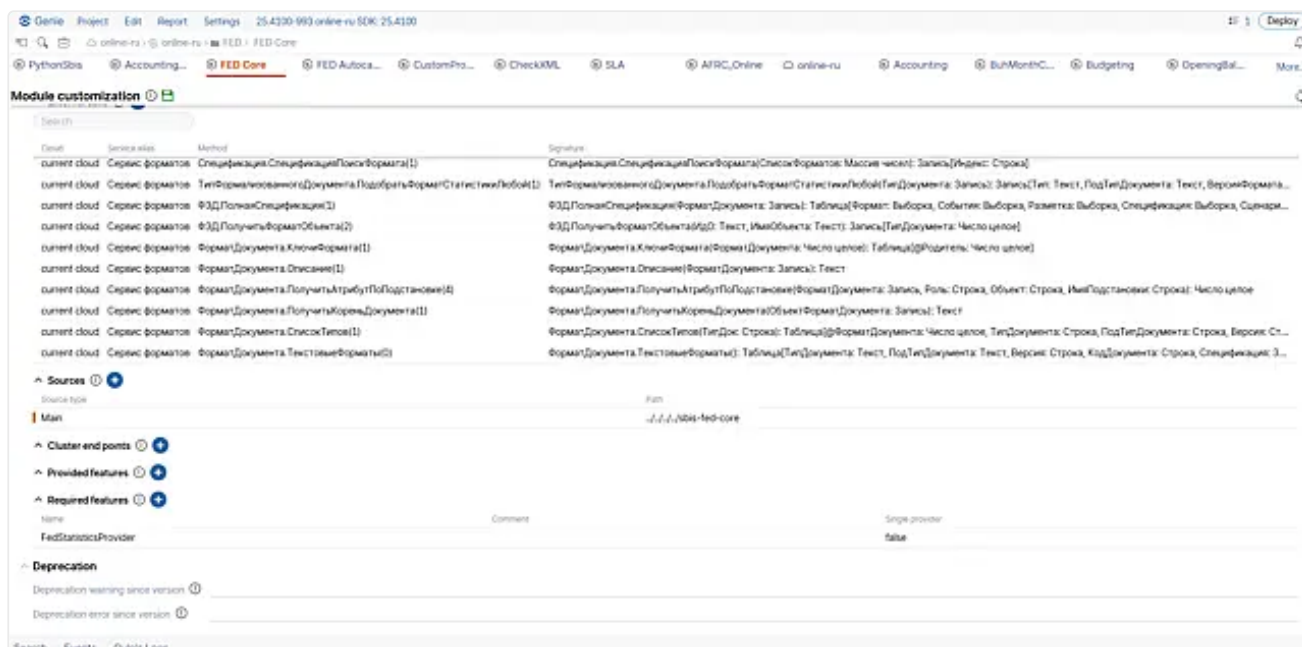
Пути к исходникам

При работе с модулями на C++ возникает две проблемы:

- исходники размещены за пределами модуля;
- необходима компиляция исходного кода.

Так как мы управляем поставкой исходников с помощью Module Manager, возникает задача генерации корневого файла [CMakeLists.txt](#) под проект, в который необходимо добавить все исходники.

Чтобы указать пути к исходникам модуля относительно корня, используется раздел **Sources**. Для одного модуля может быть задано несколько каталогов с исходниками.



```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <bl_module ...>
3     ...
4     <source_directory path="../../../../sbis-fed-core" type="main"/>
5 </bl_module>

```

Исходники делятся на три типа:

- **Main** - исходники библиотеки модуля, путь до директории с файлом CMakeLists.txt, который содержит проект библиотеки модуля.
Если у модуля есть библиотека, то для нее должен быть задан только один источник. Если библиотеки нет - источник может отсутствовать.
- **Resource** - исходники дополнительных библиотек ресурсов, путь до директории с файлом CMakeLists.txt, который содержит проект библиотеки ресурса модуля.
- **Test** - исходники тестов, путь до директории с файлом test.entry. Указывается, если у модуля есть юнит-тесты.

Устаревание модулей

В разделе Deprecation задается версия модуля, с которой он считается устаревшим.

- **Deprecation warning since version** - версия, начиная с которой модуль считается устаревшим, и нужно начать принимать меры, чтобы прекратить его использование. Если описание модуля оказывается в проекте с версией, начиная с заданной в этом поле, то формируется предупреждение.
- **Deprecation error since version** - версия, начиная с которой модуль считается устаревшим настолько, что его использование полностью недопустимо. Если описание модуля оказывается в проекте с версией, начиная с заданной в этом поле, то формируется ошибка.

Справочник объектов

Серверное кэширование вызова методов БЛ

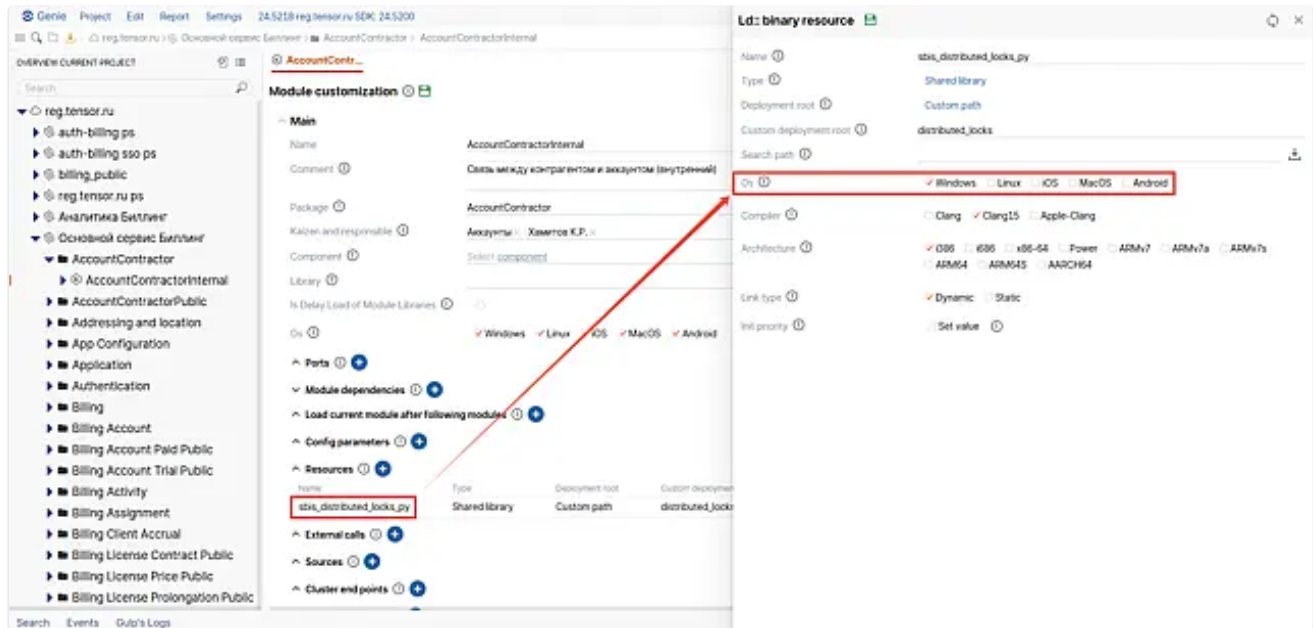
Это сохранение результатов вызова метода с заданными значениями аргументов. Используется в случае частого вызова метода, результат выполнения которого изменяется редко или не изменяется вообще. К таким методам можно отнести чтение изображений или файлов. Напротив, списочные методы, например, выводющие последние новости, кэшировать не стоит вовсе, т.к. данные могут меняться ежеминутно.

С точки зрения [оптимизации приложения](#), кэширование позволяет ускорить работу метода и снизить нагрузку на серверы БД и БЛ. Для исключения ситуации чтения устаревших данных рекомендуется дополнительно к данной настроить опцию [инвалидации кэша](#).

Платформозависимые элементы проекта

Некоторые элементы дерева проекта могут быть платформозависимыми.

Платформозависимый [сервис приложения](#) (s3srv-файл) существует не под все операционные системы, соответственно, в описании сервиса в Genie, в атрибуте Os активированы не все флаги, в зависимости от операционной системы.



Платформозависимый сервис нужно разворачивать в составе приложения или использовать в качестве родительского сервиса для других сервисов только в случае, если развертывание ведется на одной из поддерживаемых операционных систем.

[Модуль](#) также может быть платформозависимым. При развороте сервиса на операционную систему, не указанную в описании модуля, такой модуль будет отсутствовать.

Платформозависимые [ресурсы](#) имеют более богатую конфигурацию, т.е. такие ресурсы могут быть нужны не только под определенную операционную систему, но и под определенные компилятор сборки и архитектуру. Это значит, что такой ресурс должен быть упакован в состав приложения, когда сборка осуществляется под набор конфигураций, соответствующих этой маске, например:

